Tutorial -2

Intro to Deep Learning

- Advaith
- Patanjali
- Ayan

Neural Networks

Linear Transformations all the way

$$Y = wX + b$$

Activation Functions

Sigmoid

Tanh

Relu

Softmax

Loss

Cross Entropy - Classification
Negative Log Likelihood - Classification
Mean Absolute Error Loss - Regression
Mean Squared Error Loss - Regression

Gradient Descent and Back Propogation

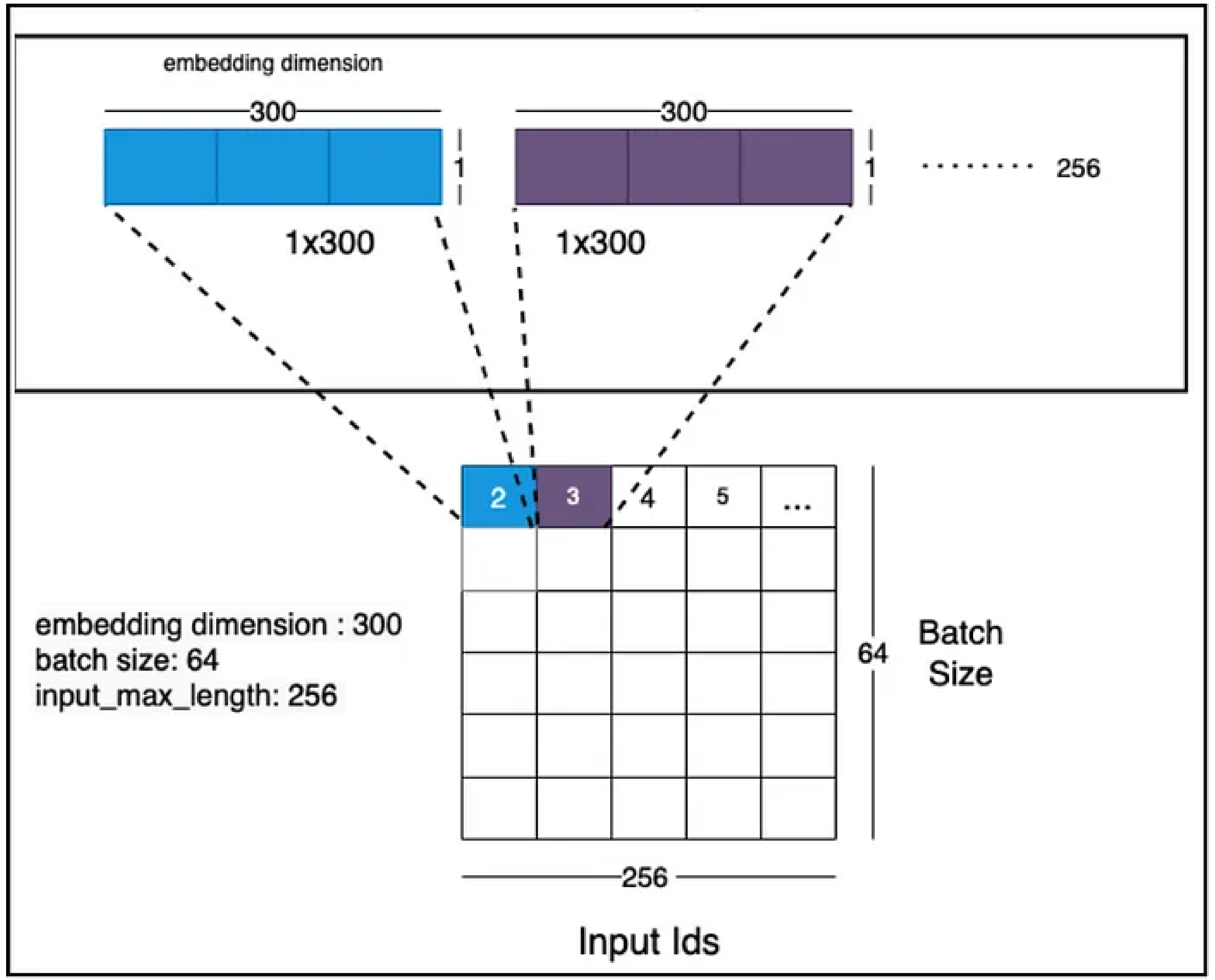
Neural POS Tagging.

Things we will see:

1. Embedding Layer
2. FFN Layers.
3. Tag Probability distribution.
4. Start and End Tokens

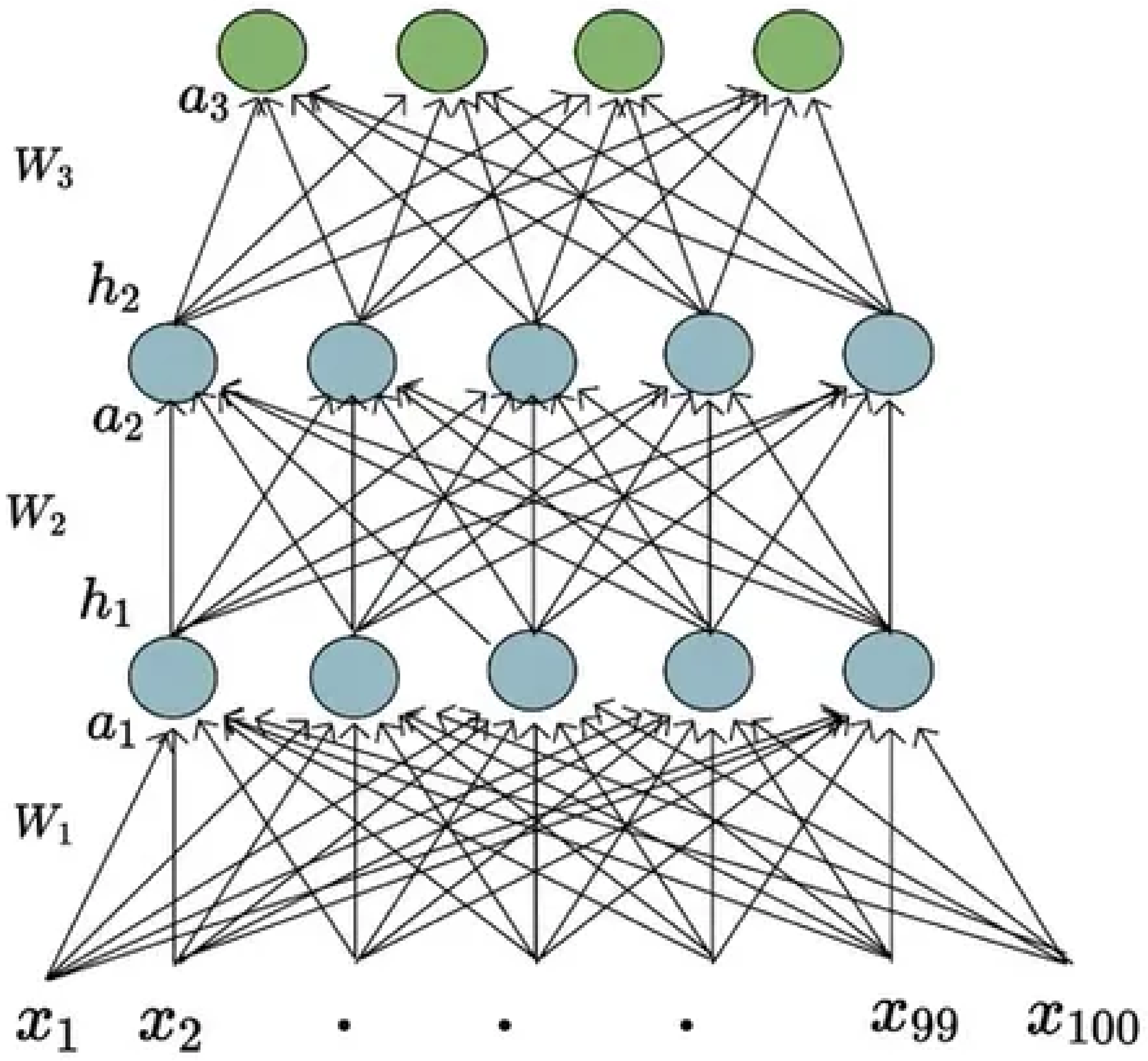
Embedding Layer.

- We need a Mathematical representation of the textual data for computational purposes.
- Simple Idea: One hot encoding.
- Can we do better? Can we learn the embeddings of the words as we train our model.
- Embedding layer is also a **Matrix**, and what is the Matrix dimension?
- PyTorch provides a Module for Embedding layer, which we can initialize with appropriate dimensions



Feed Forward Neural Networks

- Input Dimension?
  (p + 1 + s) x Embed_dim
- Hidden Layers (Look at it as deeper and abstract representations == More trainable parameters)
- Question: What are the dimensions of the first Weight Matrix?
- Output Layer and what are its dimensions? - **Logits**

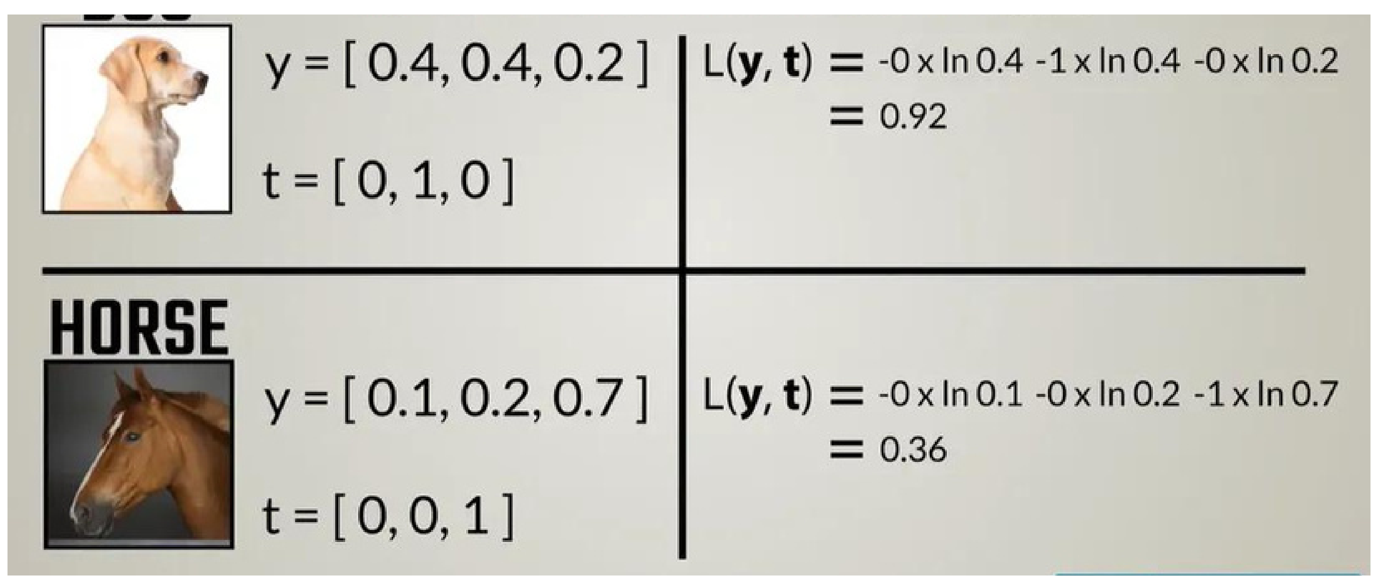


Output Layer.

- The last layer in the architecture is ideally the output layer, what should be it’s dimension?

- To get the prediction, we do a SOFTMAX or simply use a Cross-Entropy Loss function.

$$\text{Cross-Entropy} = L(\mathbf{y}, \mathbf{t}) = -\sum_i \mathbf{t}_i \, \ln \mathbf{y}_i$$



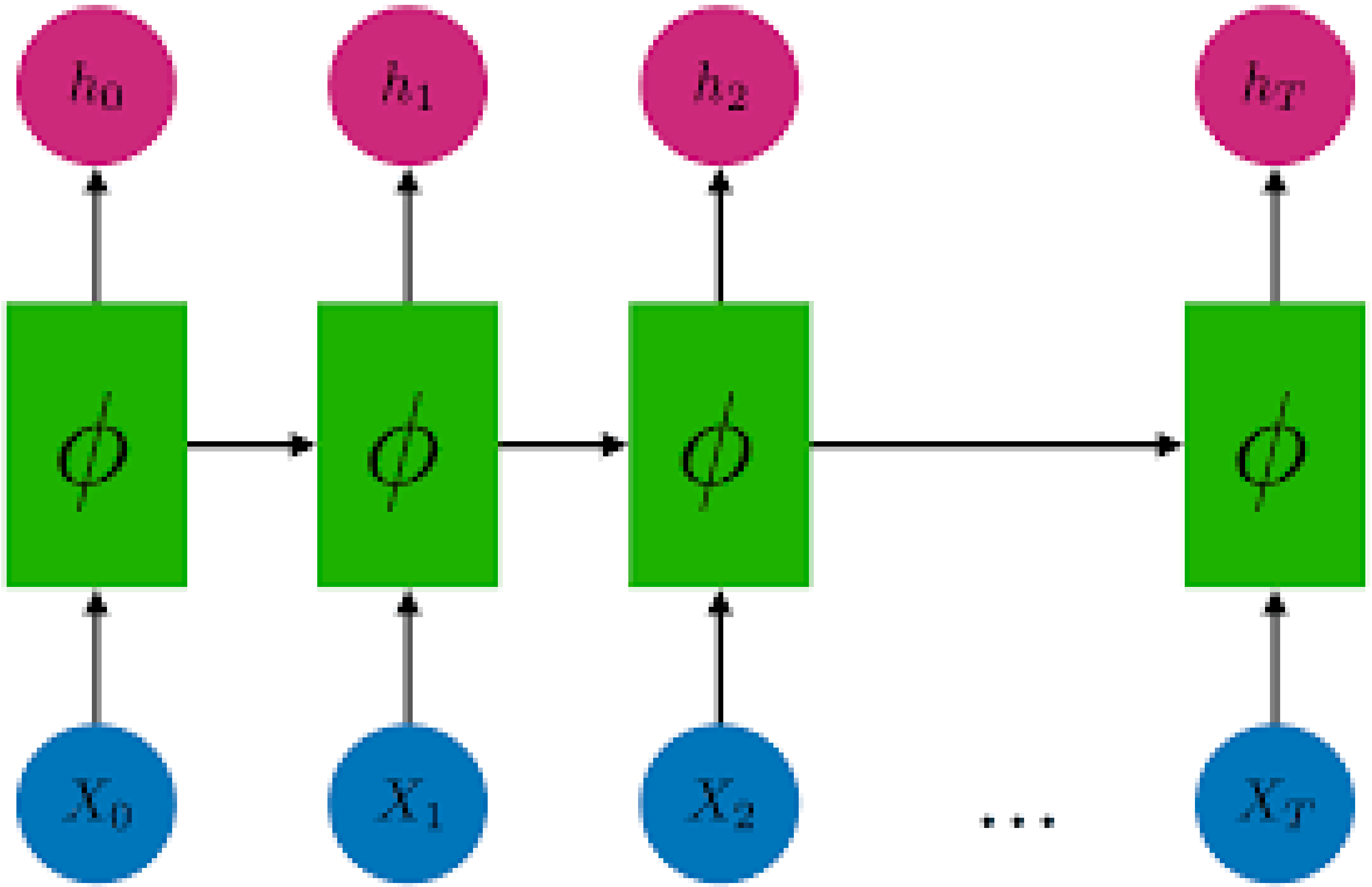
Start and End tokens and OOV?

PyTorch

- Tensors
- Datasets
- Modules
- Making a module class
- Dataloader
- Vocabularies
- Training-Validation loop
- Saving-Loading
- Using GPUs / Device Agnostic Code

Notebook Link

Inference in an RNN



Resources

How to Create a Neural Network

https://www.3blue1brown.com/topics/neural-networks

Pytorch Docs and Tutorials

Jurafsky and Martin

Understanding LSTMs